Unit 5

5.1 Compiler: Basic compilers function

- A compiler is a translator that translates the language at the highest level into the language of the computer.

- A developer writes high-level language and the processor can understand machine language.

- The compiler is used to display a programmer's errors.

- The main objective of the compiler is to modify the code written in one language without altering the program's meaning.

- When you run a programme that is written in the language of HLL programming, it runs in two parts.

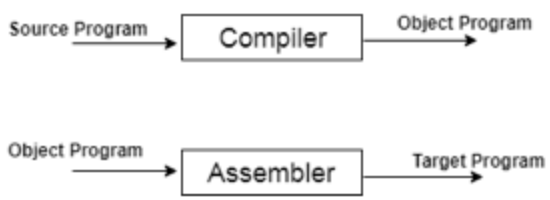


Fig 1: Execution process of Compiler

Key takeaway

A compiler is a translator that translates the language at the highest level into the language of the computer.

5.2 Phase of compilers

The method of compilation includes the sequence of different stages. Each phase takes a source programme in one representation and produces output in another representation. From its previous stage, each process takes input.

The various stages of the compiler take place:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Code Generation

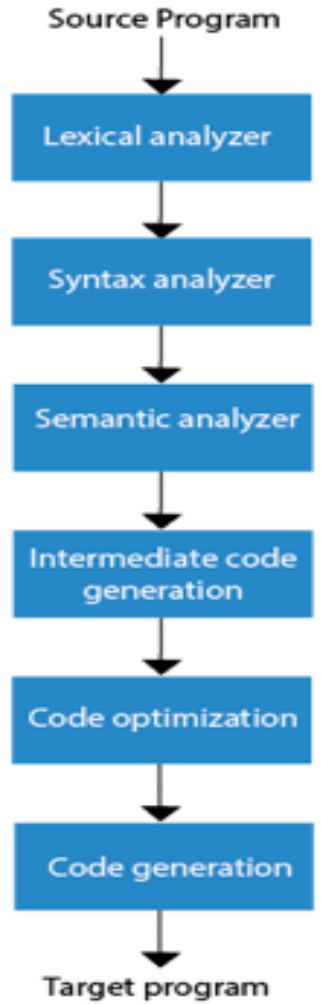


Fig 2: Phases of compiler

1. **Lexical Analysis:** The first phase of the compilation process is the lexical analyzer phase. As input, it takes source code. One character at a time, it reads the source programme and translates it into meaningful lexemes. These lexemes are interpreted in the form of tokens by the Lexical analyzer.
2. **Syntax Analysis:** Syntax analysis is the second step of the method of compilation. Tokens are required as input and a parse tree is generated as output. The parser verifies that the expression made by the tokens is syntactically right or not in the syntax analysis process.
3. **Semantic Analysis:** The third step of the method of compilation is semantic analysis. This tests whether the parse tree complies with language rules. The semantic analyzer keeps track of identifiers, forms, and expressions of identifiers. The output step of semantic analysis is the syntax of the annotated tree.

4. **Intermediate Code Generation:** The compiler generates the source code into an intermediate code during the intermediate code generation process. Between high-level language and machine language, intermediate code is created. You can produce the intermediate code in such a way that you can convert it easily into the code of the target computer.
5. **Code Optimization:** An optional step is code optimization. It is used to enhance the intermediate code so that the program's output can run quicker and take up less space. It eliminates redundant code lines and arranges the sequence of statements to speed up the execution of the programme.
6. **Code Generation:** The final stage of the compilation process is code generation. It takes as its input the optimized intermediate code and maps it to the language of the target computer. The code generator converts the intermediate code to the required computer's machine code.

5.3 Lexical Analysis - Role of Finite State Automata in Lexical Analysis

- To identify patterns, finite state machines are used.
- The Finite Automated System takes the symbol string as an input and modifies its state accordingly. When a desired symbol is found in the input, then the transformation happens.
- The automated devices may either switch to the next state during the transition or remain in the same state.
- FA has two states: state approve or state deny. When the input string is processed successfully and the automatic has reached its final state, it will approve it.

The following refers of a finite automatic:

Q: finite set of states

∑: finite set of input symbol

q0: initial state

F: final state

δ: Transition function

It is possible to describe transition functions as

$\delta: Q \times \sum \rightarrow Q$

The FA is described in two ways:

1. DFA
2. NDFA

DFA

DFA stands for Deterministic Finite Automata .Deterministic refers to computational uniqueness. In DFA, the character of the input goes to only one state. The DFA does not allow the null shift, which means that without an input character, the DFA does not change the state.

DFA has five tuples $\{Q, \sum, q0, F, \delta\}$

Q: set of all states

$\sum$: finite set of input symbol where $\delta: Q \times \sum \rightarrow Q$

q0: initial state

F: final state

δ: Transition function

NDFA

NDFA applies to Finite Non-Deterministic Automata. It is used for a single input to pass through any number of states. NDFA embraces the NULL step, indicating that without reading the symbols, it can change the state.

Like the DFA, NDFA also has five states. NDFA, however, has distinct transformation features.

NDFA's transition role may be described as:

$\delta: Q \times \sum \rightarrow 2^{Q}$

Key takeaway:

- To identify patterns, finite state machines are used.
- FA has two states: state approve or state deny.
- In DFA, the character of the input goes to only one state.
- It is used for a single input to pass through any number of states.

5.4 Design of lexical analyzer, Data structure used

- Lexical Analyzer Generator introduces a tool called Lex, which enables a lexical analyzer to be described by defining regular expressions to define token patterns.

- The lex tool input is the lex language

- A programme written in the lex language can compile and generate a C code named lex.yy.c through the lex compiler, i.e.

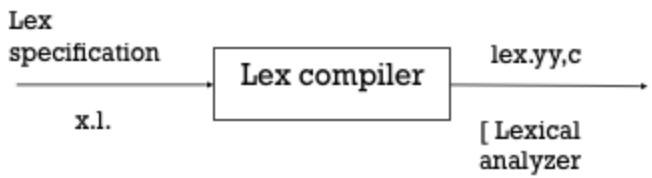


Fig 3: Lex compiler

Now, as always, C code is compiled by the C compiler and produces a file called a.out, i.e.

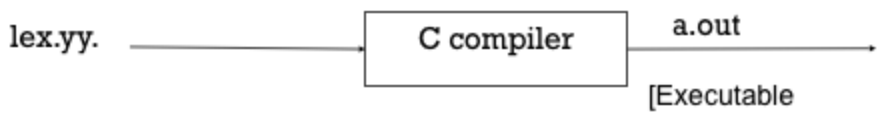


Fig 4: C compiler

The output of the C compiler is a working lexical analyser that can take an input character stream and generate a token stream, i.e.,

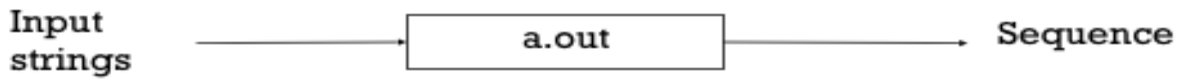


Fig 5: Generator of lexical analyzer

- Lex is a lexical analyzer generating programme. It is used to produce the YACC parser. A software that transforms an input stream into a sequence of tokens is the lexical analyzer. It reads the input stream and, by implementing the lexical analyzer in the C programme, produces the source code as output.

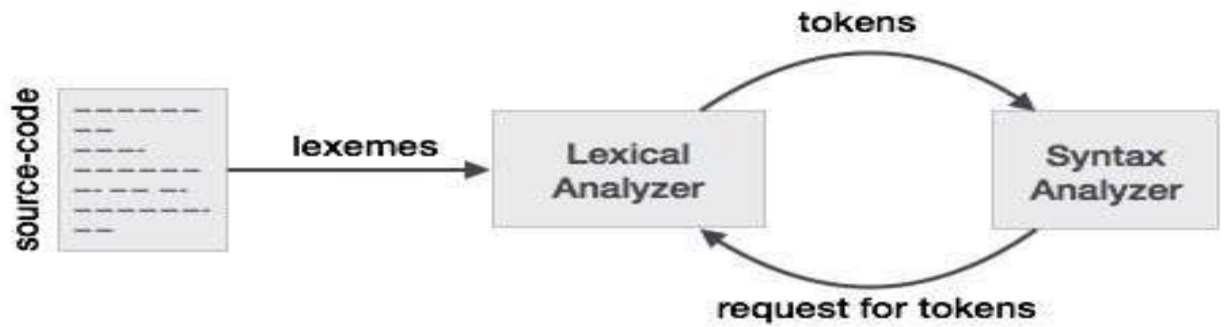


Fig 6: Lexical analyzer generator

If a token is considered invalid by the lexical analyzer, it produces an error. The lexical analyzer works with the syntax analyzer in close collaboration. It reads character streams from the source code, checks for legal tokens, and, when necessary, passes the information to the syntax analyzer.

Structure of lex program

A Lex program has the following form

Declarations

%%

Transition rules

%%

Auxiliary functions

Variable declarations, manifest constants (LT, LE, GT, GE) and standard definitions are included in the declaration section.

The basic type of rule for transition is: pattern{Action}

Where the pattern is a regular expression and action determines what action should be taken when it matches the pattern.

Second, the auxiliary function includes whatever extra features are used in the actions.

You can separately compile these functions and load them with the lexical analyzer.

LEX Program for Tokens

```
% {

ID, NUMBNER, RELOP, IF, THEN,

ELSE

% } | | declaration of variables

Delim [ space\t\n ]

Ws [ delim ] +

Letter [A - Z - a - z ]

Digit [ 0 - 9 ]

Id letter (letter / digit)

Number { digit } + ( \ . { digit } + ) ?

E [ +- ] ?  {digit +} +)?

%%

*transition rule * |

{ ws } { No Action }

If { return (IF) ; }

{ id } { return (ID) ; Install - ID () ; }

1 { number } { return (number); Install-num (); }
```

“<” { return (RELOP, LT) }

“<=” { return (RELOP, LE) }

“=” { return (RELOP, EQ) }

%%

|* Auxiliary Section *|

Install-ID ()

{

=

=

Y

Install-num ()

{

=

=

}

5.5 Syntax Analysis - Role of context Free Grammar in Syntax analysis

Context free grammar is a formal grammar which is used in a given formal language to generate all possible strings.

Four tuples can be described by Context free grammar G as:

G= (V, T, P, S)

Where,

G - grammar

T - finite set of terminal symbols.

V - finite set of non-terminal symbols

P - set of production rules

S - start symbol.

In CFG, to derive the string, the start symbol is used. You may extract a string by replacing a non-terminal on the right side of the output repeatedly until all non-terminals are replaced with terminal symbols.

Example:

$L= \{wcw^R \mid w \in (a, b)^*\}$

Production rule :

1. S → aSa
2. S → bSb
3. S → c

Now, verify that you can derive the abbcbba string from the given CFG.

1. S ⇒ aSa
2. S ⇒ abSba
3. S ⇒ abbSbba
4. S ⇒ abbcbba

By recursively applying the production S → aSa, S → bSb and finally applying the production S → c, we get the abbcbba series.

Key takeaway:

- In CFG, to derive the string, the start symbol is used.
- Context free grammar is a formal grammar which is used in a given formal language to generate all possible strings.

5.6 Study of LEX & YACC

Functions of lex:

- Firstly, in the language of Lex, the lexical analyzer produces a lex.1 programme. The Lex compiler then runs the lex.1 programme and generates the lex.yy.c C programme.

- Finally, the programmer C runs the programme lex.yy.c and generates the object programme a.out.

- A.out is a lexical analyzer which converts an input stream into a token sequence.

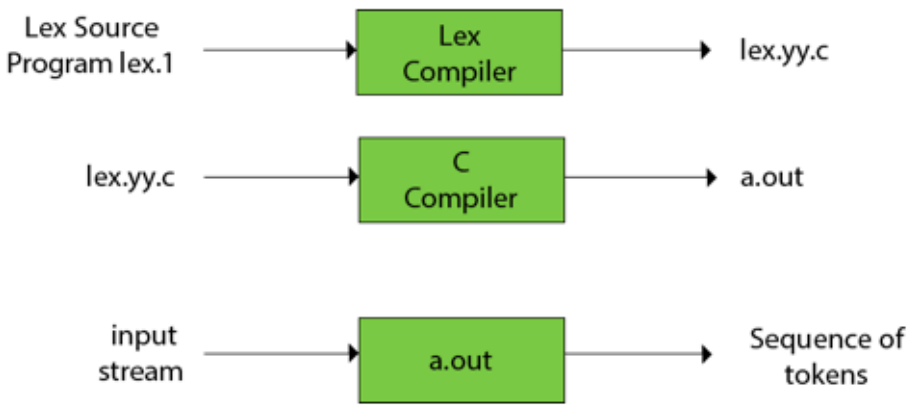


Fig 7: Lex compiler

Lex file format:

By percent percent delimiters, a Lex programme is divided into three parts. The structured sources for Lex are as follows:

{ definitions }

%%

{ rules }

%%

{ user subroutines }

Definitions include constant, variable and standard meaning declarations.

The **rules** describe the form statement p1 {action1} p2 {action2}....pn {action} p2 form p1....pn {action}

Where pi describes the regular expression and **action1** describes the behaviour that should be taken by the lexical analyzer when a lexeme matches the pattern pi.

Auxiliary procedures required by the acts are **user subroutines**. It is possible to load the subroutine with the lexical analyser and compile it separately.

YACC

- YACC stands for Compiler Compiler Yet Another.
- For a given grammar, YACC provides a method to generate a parser.
- YACC is a software designed to compile the grammar of LALR(1).
- It is used to generate the source code of the language syntactic analyzer provided by LALR (1) grammar.
- The YACC input is a rule or grammar, and a C programme is the output.

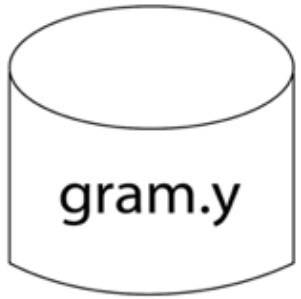
Some points about YACC are as follows:

Input: A CFG- file.y

Output: A parser y.tab.c (yacc)

- The output file "file.output" includes tables of parsing.
- There are declarations in the file 'file.tab.h'.
- Yyparse was named by the parser ().
- In order to get tokens, Parser expects to use a function called yylex ().

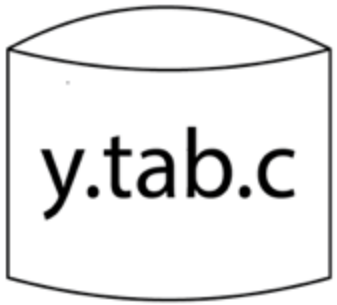
The following is the essential operational sequence:



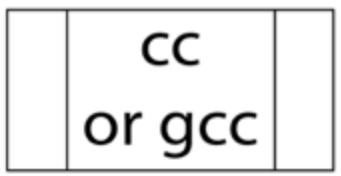
In YACC format, this file contains the desired grammar.



The YACC software is seen.



It is the YACC-generated c source software.



C - compiler



Executable file to decode grammar defined in gram.YY.

Key takeaway:

- In the language of Lex, the lexical analyzer produces a lex.l programme.
- The Lex compiler then runs the lex.
- The YACC input is a rule or grammar, and a C programme is the output.
- For a given grammar, YACC provides a method to generate a parser.

5.7 Benefits of Interpretation

- Cross-Platform - We immediately share the source code in interpreted languages, which can run on any machine without any system incompatibility issues.

- Easier To Debug - Interpreters make code debugging easier because they read the code line by line and return the error message right away. In addition, if the client has access to the source code, they can quickly debug or alter it.

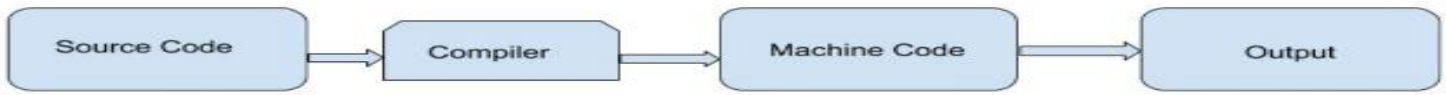
- Less Memory and Step - Interpreters, unlike compilers, do not create new distinct files. So it doesn't take up any extra memory, and we don't have to do anything extra to execute the source code because it's done on the fly.

- Execution Control - Because an interpreter reads code line by line, you can pause and edit it at any time, which is impossible with a compiled language. However, after being stopped it will start from the beginning if you execute the code again.

5.8 Overview of Interpretation

An interpreter is a programme that directly executes high-level language instructions rather than translating them to machine code. We can run a programme in two methods in programming. Compilation is the first step, followed by an interpreter. Using a compiler is the most usual method.

Interpreters, like compilers, perform the same function. It can also convert high-level languages to low-level languages.



How Compiler Works



How Interpreter Works

Fig 8: Interpreter

However, unlike a compiler, an interpreter analyses the source code line by line and warns you if there is a mistake at the same time, making it easier to debug but slower than a compiler.

Strategies of an Interpreter

It can be used in three different ways:

- Directly execute the source code and generate the output.
- The source code should be translated into intermediate code, which should then be executed.
- Creating precompiled code with an internal compiler. Then run the code that has already been precompiled.

Role of Interpreter

- During RUN Time, the interpreter converts the source code line by line.
- Interpret converts a programme written in a high-level language to a machine-level language in its entirety.
- The interpreter allows the programme to be evaluated and modified while it is running.
- The programme was analyzed and processed in a shorter amount of time.
- When compared to the compiler, programme execution is comparatively slow.

5.9 The Java Language Environment

Java is a sophisticated general-purpose programming language that was first released in 1996 and has been around for almost 23 years. The Java programming environment is made up of the following components:

- Java language - programmers use to create the application
- Java virtual machine - The application's execution is controlled by this variable.
- Java ecosystem - delivers added value to programmers who use the programming language.

Java Language

- It's a human-readable language that's quite simple to understand and write (albeit a bit verbose at times).
- It's a class-based, object-oriented system.
- Java is meant to be simple to understand and teach.
- There are numerous distinct Java implementations, both proprietary and open source, available.
- The Java Language Specification (JLS) specifies how a Java application shall behave when it confirms.

Java virtual memory

The Java Virtual Machine is a program that creates a runtime environment for Java programs to run in. If a supporting JVM is not available, Java programs will not run.

When using the command line to launch a Java program, such as —

Java <arguments> <program name>

The operating system would start the JVM as a process, then run the program in the newly started (and empty) Virtual Machine.

Java source files are not accepted as input by JVM. The javac application converts the java source code to bytecode first. Javac accepts source files as input and produces bytecode in the form of class files ending in.class.

The JVM interpreter then steps through these class files one by one, interpreting them and executing the program.

The Just-in-Time (JIT) compiler is a key feature of the JVM. The runtime behavior of programs shows certain fascinating patterns, according to research conducted in the 1970s and 1980s. Some sections of code are executed significantly more frequently than others.

The HotSpot JVM (originally released by Sun in Java 1.3) finds the "hot methods" (often called) and the JIT compiler turns them directly into machine code, avoiding the compilation of source code into bytecode.

Java ecosystem

Java has gained widespread acceptance as a reliable, portable, and high-performing programming language. The enormous amount of third-party libraries and components created in Java is one of the key reasons for Java's success. It's rare nowadays to come across a component that doesn't have a Java connector. There is a Java connector for everything from standard MySQL to NoSQL, monitoring frameworks, and network components.

Key takeaway

Java is a platform as well as a programming language.

Java is a high-level programming language that is also robust, object-oriented, and secure.

Java is a sophisticated general-purpose programming language that was first released in 1996 and has been around for almost 23 years.

5.10 Java Virtual Machine

JVM (Java Virtual Machine) is a program that gives us the flexibility to execute Java programs. It provides a runtime environment to execute the Java code or applications.

This is the reason why Java is said to platform independent language as it requires JVM on the system it runs its Java code.

JVM is also used to execute programs of other languages that are compiled to Bytecode.

JVM can also be describe as:

1. A **specification** where JVM working is pre defined.
2. An **implementation** like JRE (Java Runtime Environment) is required.
3. **Runtime Instance** means wherever a program is run on command prompt using java command, an instance of JVM is created.

Significance of JVM

JVM is used for two primary functions

1. It allows Java programs to use the concept of "Write Once and Run Anywhere".
2. It is utilized to manage and optimize memory usage for programs.

Architecture of JVM

Java applications require a run-time engine which is provided by JVM. The main method of Java program is called by JVM. JVM is a subsystem of JRE (Java Runtime Environment).

Let's see the internal architecture of JVM as shown in the figure below:

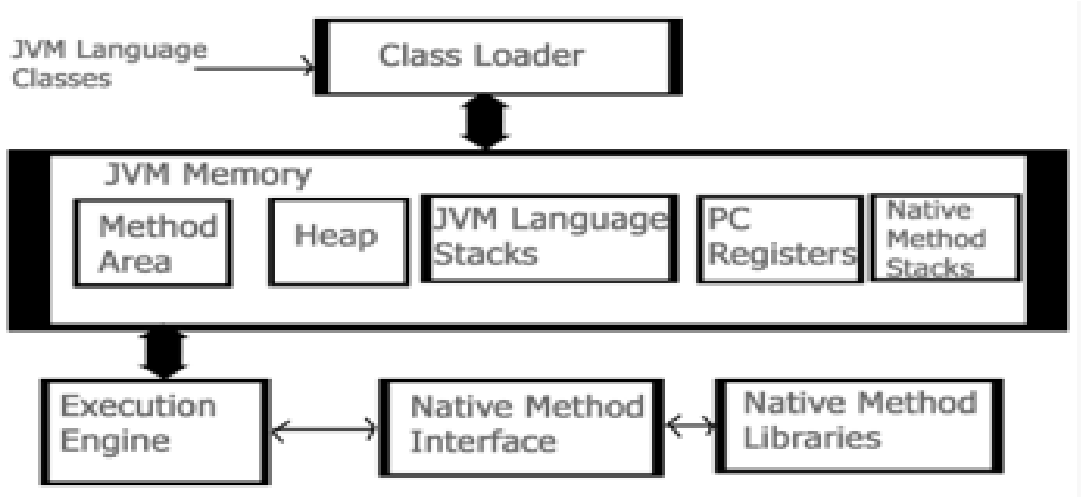


Fig 9: Architecture of JVM

1. **Class Loader:** This part is used to load class files. There are three major functions of class loader as Loading, Linking and Initialization.
2. **Method Area:** Structure of class like metadata, the constant runtime pool, and the code for methods are stored method area.
3. **Heap:** Heap is the memory where all the objects, instance variables and arrays are stored. This is a shared memory which is accessed by multiple threads.
4. **JVM Language Stacks:** These hold the local data variables and their results. Each thread has their own JVM stack used to store data.
5. **PC Registers:** The currently running IVM instructions are stored in PC Registers. In Java, every thread has its their own PC register.
6. **Native Method Stacks:** It hold the instruction of native code depending on their native library.
7. **Execution Engine:** This engine is used to test the software, hardware or complete system. It doesn't contain any information about the tested product.
8. **Native Method Interface:** It is a programming interface. It allows Java code to call by libraries and native applications.
9. **Native Method Libraries:** It consists of native libraries which are required by execution engine.

5.11 Types of Errors

Experiencing various forms of programming failures is an important part of the development process. The greatest developers get adept at navigating and swiftly resolving the faults they cause.

The following are the seven most common programming errors:

Syntax Error

Computer languages, like human languages, have grammar rules. However, whereas people can communicate with less-than-perfect grammar, computers are unable to ignore faults, such as syntax errors.

Let's imagine the right syntax for printing something is print('hello'), and one of the parentheses is mistakenly left out during writing. A syntax error will occur, preventing the programme from functioning.

As your knowledge of the programming language grows, you will make fewer syntax errors. Being aware of them early on is the simplest method to prevent them from causing you difficulties. Many text editors or integrated development environments (IDEs) will notify you about syntax problems as you type.

Runtime Error

Runtime errors occur when a user runs your programme. The code may execute successfully on your machine, but it may be configured differently on the website, or it may be interacted with in a way that causes a runtime error.

If you used something like params[:first name] to capitalise the first letter of a name that was entered into a form.

If the form was delivered without a first name, this would be broken.

Runtime problems are particularly aggravating since they affect your end user immediately. Many of these additional mistakes will occur while you're working on the code at your PC. These problems occur while the system is functioning and can prevent someone from accomplishing their goals.

Make sure you have good error reporting in place to catch any runtime failures and open new bugs in your ticketing system automatically. Try to learn from each bug report so that you can avoid making the same mistake in the future.

Using frameworks and community-maintained code is a fantastic approach to reduce these types of problems because the code is used in a variety of projects and has previously encountered and resolved several issues.

Compilation Error

Compilation is required in several programming languages. Compilation is the process of converting your high-level language into a lower-level language that the computer can comprehend. When the compiler doesn't know how to put your code into lower-level code, it causes a compilation or compile-time error.

If we were compiling print('hello') in our syntax error example, the compiler would pause and inform us it doesn't know how to translate this to a lower-level language since it expected a) after the '.

You won't be able to test or start your software if it contains a compile-time fault.

You'll get better at avoiding these with time, just like you'll get better at avoiding syntax errors, but in general, the best thing you can do is obtain early feedback when they arise.

At the same moment, compilation takes place across all of your project's files. It can be intimidating if you've made a lot of changes and notice a lot of compiler warnings or problems. You'll obtain the input you need sooner if you run the compiler frequently, and you'll be able to pinpoint where the problems are more readily.

Logic Error

The most difficult to trace down are logic flaws. Everything appears to be in working order; you have simply configured the computer to perform the incorrect thing. Although the software is technically valid, the results will not be what you intended.

You'd have a logic problem if you didn't check the requirements first and developed code that returned the oldest user in your system when you wanted the newest.

A well-known example occurred in 1999, when NASA lost a spacecraft due to errors made by English and American units. The software was written in one way, but it required to function in another.

Show your tests to the product manager or product owner as you create them to ensure that the logic you're about to write is valid. Someone more familiar with the company might have seen that you aren't indicating that the newest user is needed in the example above.

Interface Error

When there is a discrepancy between how you intended your application to be used and how it is actually used, interface issues occur. The majority of software adheres to a set of rules. You could get an interface error if the input your software gets does not meet the standards.

For example, if you have an API that requires specified parameters to be set and those parameters are not set, an interface error may occur.

Interface errors, if not handled properly, will appear to be an error on your end when they are actually an error on the caller's end. Both parties may become frustrated as a result of this.

The easiest approach to say, "Hey, you haven't provided us what we need to process this request," is to have explicit documentation and catch these errors to communicate back to the caller in a helpful way. This can help you save money on support and keep your customers pleased because they'll know exactly what they need to repair.

If you don't detect these mistakes and transmit them back to the caller, they'll show up in your reporting as runtime errors, and you'll wind up overprotecting yourself.

Arithmetic Error

Arithmetic errors are similar to logic errors, except they involve mathematics. When conducting a division equation, a common example is that you cannot divide by 0 without generating an error. Few individuals would write 5 / 0, but you might not realize that the size of something in your system could occasionally be zero, resulting in this error.

Ages.max / ages.min could return an error if either ages.max or ages.min were zero.

As we've seen, arithmetic errors can lead to logic problems or, in the instance of divide by zero, even run-time faults.

Having functional tests that always include edge-cases such as zero or negative values is a great method to halt arithmetic errors in their tracks.

Resource Error

Your programme will be given a set amount of resources to run on the computer on which it is executing. A resource error might occur if something in your code causes the computer to try to allocate more resources than it has.

You'd eventually run out of resources if you mistakenly constructed a loop that your code couldn't exit from. The while loop in this example will continuously adding new elements to an array. You will eventually run out of RAM.

```
While(true)

My_array << 'new array element'

End
```

Because the machine you're writing on is often of greater quality than the servers serving your code, resource problems might be difficult to track down. It's also difficult to simulate real-world usage on your home computer.

Having strong resource consumption reporting on your web servers will highlight code that is consuming excessive amounts of any resource over time.

Resource errors are an example of a programming error that may be better handled by the operations team rather than the developers.

You can use a variety of load-testing programmes and services to see what happens when numerous people try to run your code at the same time. Then, you can tune the testing to suit what is realistic for your application.

5.12 Debugging Procedures

Debugging is the act of finding and fixing current and potential flaws (often known as "bugs") in software code that might cause it to behave abnormally or crash. Debugging is used to detect and fix bugs or problems in software or systems to prevent them from malfunctioning. Debugging becomes more difficult when several subsystems or modules are tightly connected, as each modification in one module may cause more bugs to arise in another. Debugging a programme can take longer than programming it.

To debug a software, the user must first identify the problem, isolate the source code, and then correct it. Because knowledge of problem analysis is expected, a user of a programme must know how to fix the problem. The software will be ready to use after the bug has been repaired. Debugging tools (sometimes known as debuggers) are used to detect coding mistakes at various stages of development. They're used to recreate the error conditions, then look at the programme state at the moment to figure out what went wrong.

Programmers can follow the execution of a programme step by step by assessing the value of variables and stopping it if necessary to obtain the value of variables or reset programme variables. Some programming language packages include a debugger that can be used to check for flaws in the code as it is being written at run time.

Debugging procedure

Debugging is the process of locating and correcting faults or flaws in any application or software. This process should be completed before releasing software programmes or items into the market to ensure that they are bug-free. The steps in this procedure are as follows:

- **Identifying the error** – It saves time and prevents errors on the user's end. Detecting faults early on helps to reduce the number of errors and time spent on them.

- **Identifying the error location** – To fix the fault faster and execute the code, the exact location of the error must be identified.

- **Analyzing the error** – We must examine the error in order to determine the sort of problem or error and to limit the number of errors. Fixing one defect may cause another to appear, halting the application process.

- **Prove the analysis** – We must prove the analysis once the inaccuracy has been identified. The test cases are written using a test automation approach through the test framework.

- Cover the lateral damage – The bugs can be fixed by making the necessary adjustments and moving on to the next steps of the code or programme to correct the remaining faults.

- Fix and Validate – This is the final stage of the process, which includes checking for any new faults or changes in the software or programme, as well as executing the application.

Debugging Tools

A debugger or debugging tool is a software tool or application that is used to test and debug other programmes. It aids in the detection of code faults at various phases of the software development process. These tools examine the test run and identify any lines of code that aren't being performed. Simulators and other debugging tools allow the user to see how the operating system or any other computing device looks and behaves. The majority of open-source tools and scripting languages do not have an IDE and must be done manually.

GDB, DDD, and Eclipse are the most commonly used debugging tools.

- **GDB Tool**: This is a type of Unix programming tool. GDB is pre-installed on all Linux computers; if it isn't, the GCC compiler package must be downloaded.

- **DDD Tool**: DDD stands for Data Display Debugger, and it is a Unix tool that is used to execute a Graphic User Interface (GUI).

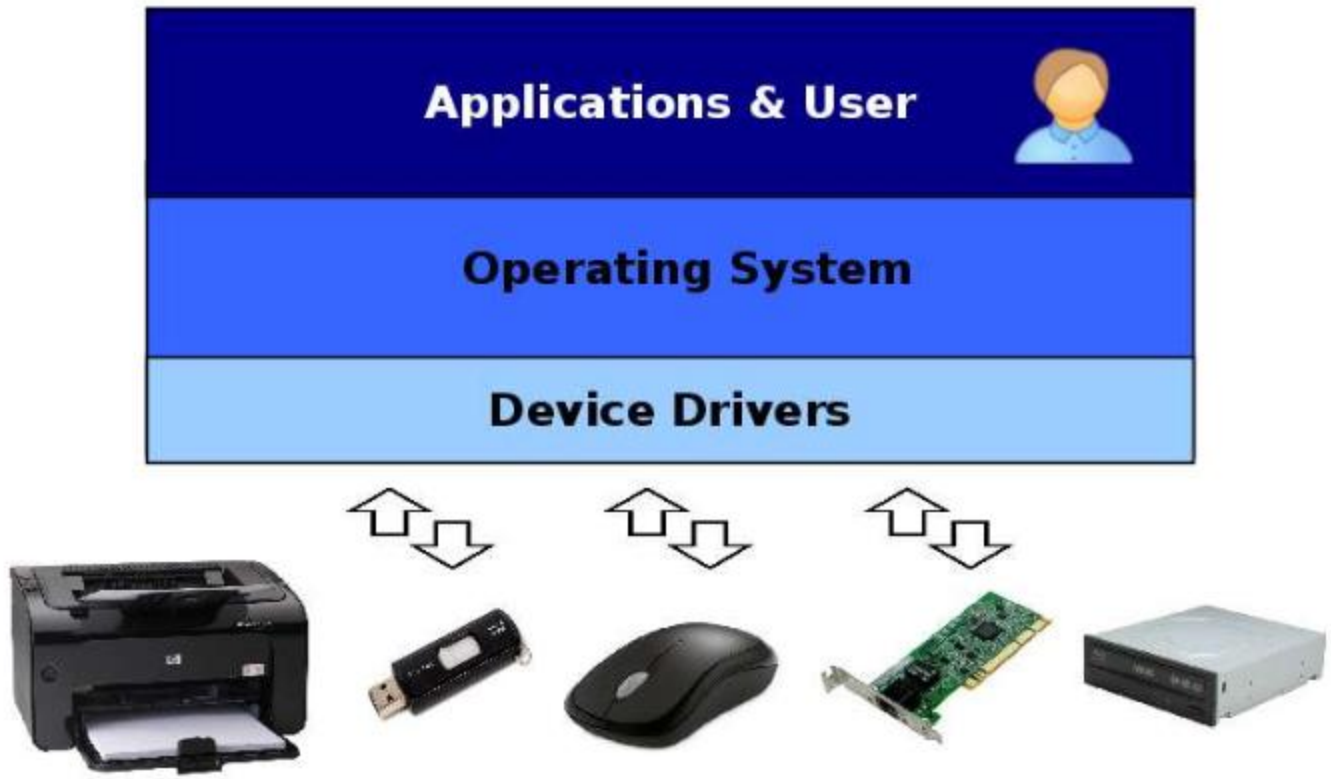
- **Eclipse**: An IDE tool combines an editor, a build tool, a debugger, and other development tools into one package. The most widely used Eclipse tool is the IDE. When compared to DDD, GDB, and other tools, it is more efficient.

Key takeaway

Debugging is a crucial approach for locating and removing the amount of errors, faults, or defects in a software. Software development is a multi-step procedure. It entails identifying the bug, tracking down the cause of the bug, and fixing the issue so that the programme is error-free.

5.13 Comparative study between device driver for UNIX & Windows

Device drivers are components of the operating system that enable software applications to manage and operate hardware devices via a specific programming interface. You'll need separate Linux, Windows, or Unix device drivers to utilize your device on different computers because each driver is specialized to a certain operating system.



Understanding the differences in how each operating system manages its drivers, the underlying driver model and architecture it employs, as well as available development tools, is the first step in driver development. For example, the Linux driver model differs significantly from that of Windows.

While Windows allows for the separation of driver and OS development and merges the two via a series of ABI calls, Linux device driver development does not rely on any fixed ABI or API, instead incorporating the driver code into the kernel. Each of these models has its own set of benefits and downsides, but if you want to provide comprehensive support for your device, you should be aware of all of them.

Divide Driver Architecture

The architecture of Windows device drivers differs from that of Linux drivers, with each having its own set of advantages and disadvantages. The fact that Windows is a closed-source OS while Linux is open-source is a major factor in the differences. The architectures of Linux and Windows device drivers will help us grasp the fundamental distinctions between the two operating systems.

Windows driver architecture

While the Linux kernel has device drivers, the Windows kernel does not. Modern Windows device drivers, on the other hand, are created using the Windows Driver Model (WDM), which fully supports plug-and-play and power management, allowing drivers to be loaded and unloaded as needed.

Requests from applications are processed by the IO manager, which converts them into IO Request Packets (IRPs), which are used to identify the request and communicate data between driver levels.

Three types of drivers are available in WDM, forming three layers:

- Filter drivers allow IRPs to be processed further if desired.
- The main drivers that implement interfaces to particular devices are function drivers.
- Bus drivers are responsible for maintaining a variety of adapters and bus controllers that host devices.

As it travels from the IO manager to the hardware, an IRP passes via these layers. Each layer can independently handle an IRP and return it to the IO manager. The Hardware Abstraction Layer (HAL) is located at the bottom and provides a common interface to physical devices.

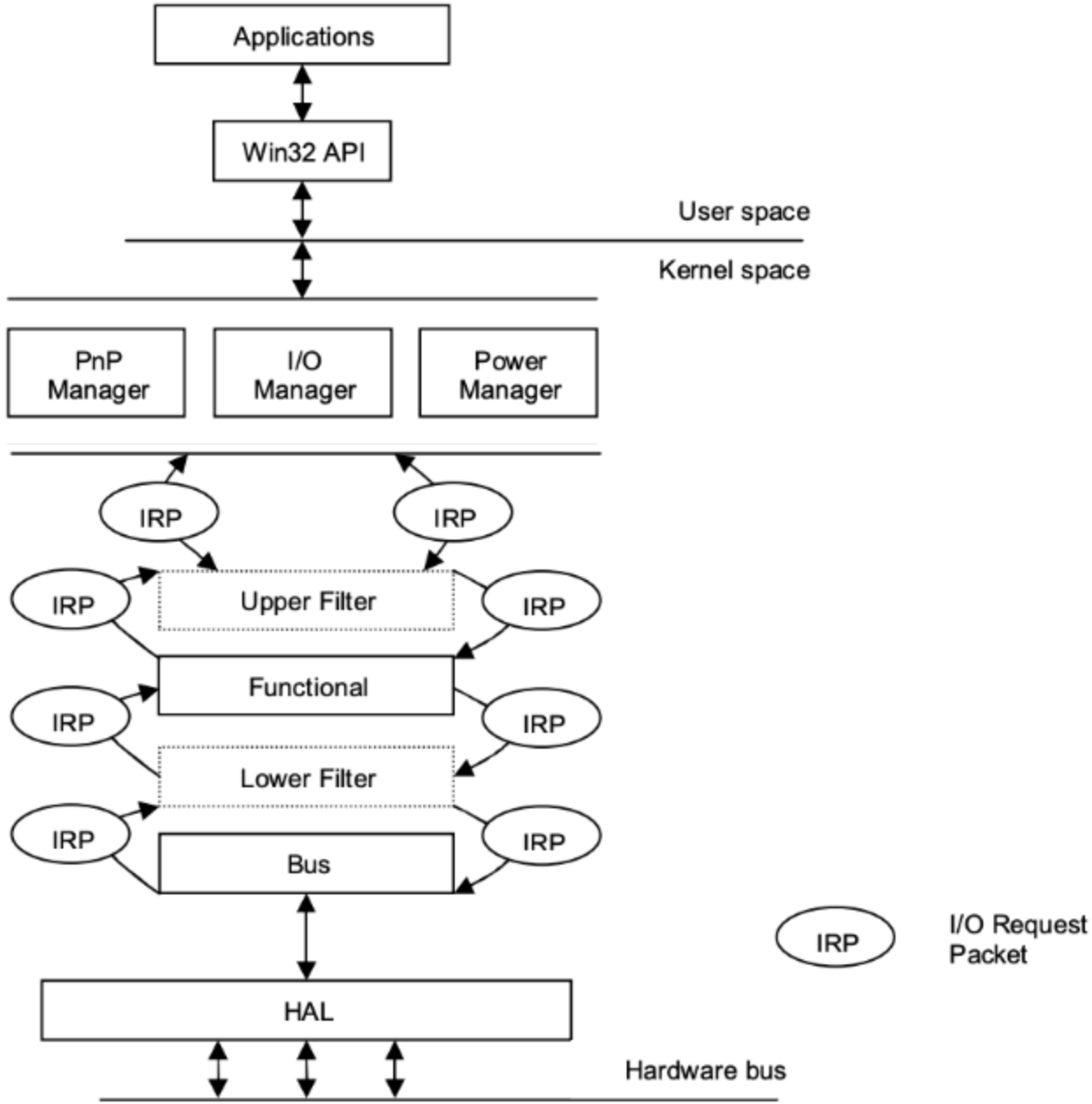


Fig 10: Windows driver architecture

Linux driver architecture

The main difference between the Linux and Windows device driver architectures is that Linux does not have a standard driver model or a clean layer separation. Typically, each device driver is implemented as a module that can be dynamically loaded and unloaded into the kernel. Linux includes plug-and-play support and power management so that drivers can use them to properly control devices, but it is not required.

Modules communicate by invoking the functions they provide and passing around arbitrary data structures. User requests originate at the filesystem or networking level, where they are translated into data structures as needed. Modules can be stacked into layers, with some modules offering a common interface to a device family, such as USB devices, and processing requests one after the other.

Three types of devices are supported by Linux device drivers:

- Byte stream devices are character devices that implement a byte stream interface.
- Block devices are filesystem hosts that conduct IO with multibyte data blocks.
- Network interfaces are devices that allow data packets to be transferred across a network.

Linux also contains a Hardware Abstraction Layer, which functions as a bridge between the device drivers and the actual hardware.

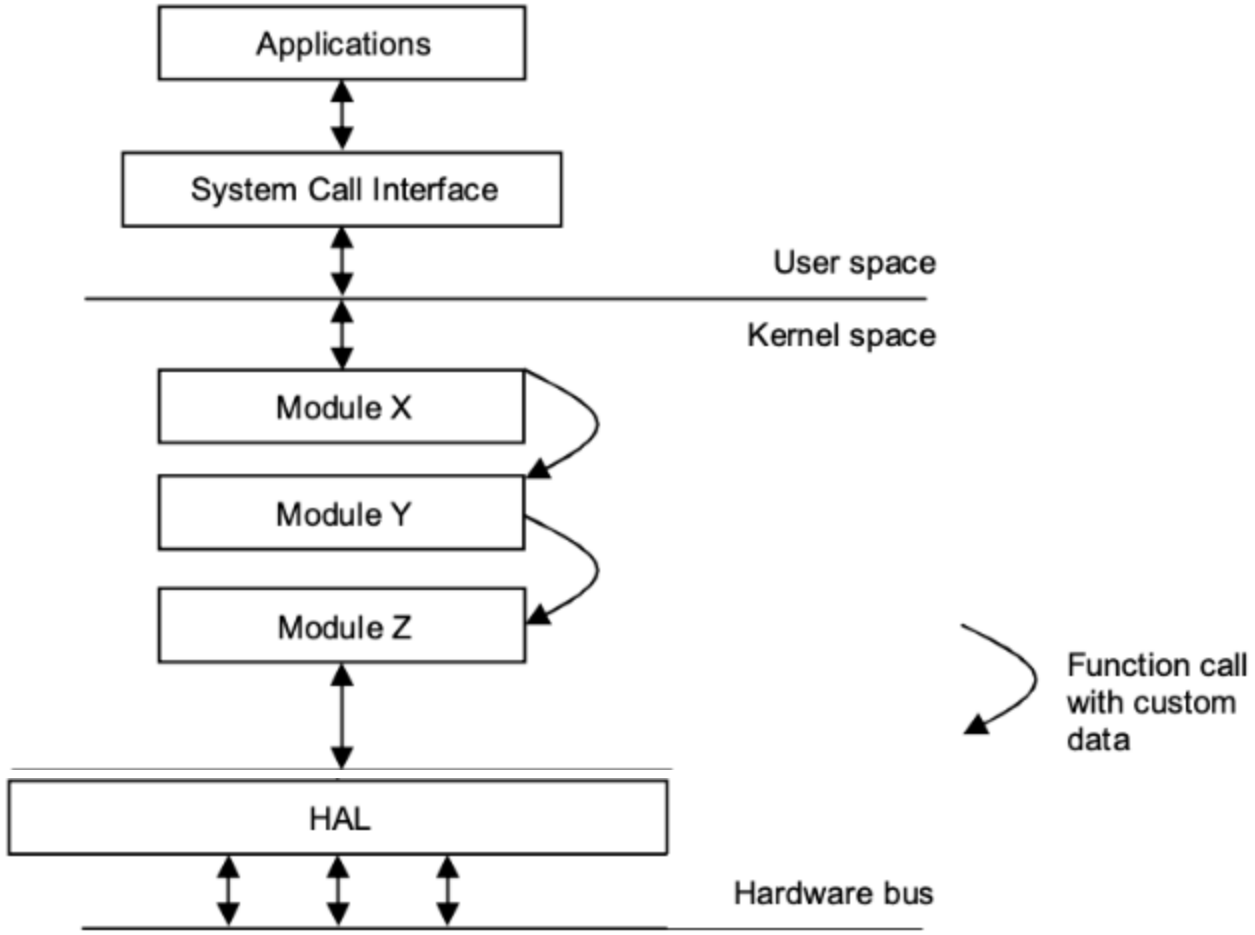


Fig 11: Linux driver architecture